AURORA DEEMED TO BE UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

BACKEND DEVELOPMENT FUNDAMENTALS

LABORATORY REPORT - LAB 1
NODE.JS AND EXPRESS SERVER

Student Name:	**J Harshini**
Course & Branch:	B.Tech CSE (AI & ML)
Academic Year & Semester:	II Year – Semester I
Class Section:	207 AI ML - 2B
Subject / Lab:	Backend Development Fundamentals – Lab 1
University:	Aurora Deemed to be University

1. Objective & Scope

The primary objective of this laboratory assignment is to understand the foundational principles of server-side web engineering using JavaScript runtimes. The practical hands-on exercises introduce students to the complete lifecycle of backend development, starting from pure Node.js low-level HTTP primitives up to high-performance, maintainable web architectures built with the Express.js framework.

- Key Learning Outcomes for this Lab:
- **Understanding Runtime vs Browser:** Appreciate how Node.js executes JavaScript on the server machine without requiring a browser window.
- **Package Lifecycle & Manifest Management:** Master initializing Node projects using npm init, understanding package.json fields, and managing the node_modules dependency tree.
- **Native HTTP Architecture:** Construct a working HTTP server using Node's core built-in http module with http.createServer().
- **Express Framework Mastery:** Install, configure, and architect a modular Express web application using CommonJS syntax (require and module.exports).
- **Routing & Request-Response Paradigms:** Implement HTTP GET endpoints returning formatted plain text, dynamic ISO date-time strings, and structured JSON payloads.
- **Error Resilience & Catch-All Handlers:** Implement professional 404 Not Found error handling for unmatched or invalid client route requests.
- **Separation of Concerns:** Decouple application routing logic (app.js) from the server network startup process (server.js).

2. Software & System Requirements

To ensure a reproducible, reliable development environment, the lab assignment was executed using the following software stack:

Software Component	Version / Platform	Role in Development
Node.js Runtime	v24.21.0 (64-bit)	Server-side JavaScript runtime engine powered by Chrome V8
npm (Package Manager)	v11.19.0	Installs and orchestrates third-party packages and runtime scripts
Express.js Framework	v5.3.0	Minimalist, flexible Node.js web application routing framework
Operating System	Microsoft Windows 11	Host operating system for local development and terminal

		execution
Testing Clients	cURL & Microsoft Edge	Terminal command-line and GUI web clients for endpoint verification

3. Project Setup and Directory Architecture

In strict adherence to the lab guidelines, a completely isolated directory named BDF-Lab-01 was established to ensure that existing NxtWave assignments remain completely undisturbed. The directory tree was intentionally architected according to industry standards for separation of concerns.

3.1 Complete Project Directory Structure

```
BDF-Lab-01/
│
├── package.json          # Project manifest, metadata, start scripts, and
dependencies
├── package-lock.json     # Lockfile locking exact versions of transitive
dependencies
├── node_modules/         # Directory containing downloaded Express library &
dependencies
├── http-server.js        # Pure Node.js HTTP server created with built-in http
module
├── app.js                # Express application configuration and route definitions
├── server.js             # Server startup entry point listening on Port 3000
├── .gitignore            # Excludes node_modules and OS cache files from version
control
├── README.md             # Complete technical documentation and quickstart
instructions
└── screenshots/          # Real output proof images from terminal and browser
    ├── terminal_setup.png
    ├── terminal_http_server.png
    ├── browser_http_server.png
    ├── terminal_express_server.png
    ├── route_root.png
    ├── route_about.png
    ├── route_time.png
    └── route_404.png
```

3.2 Step-by-Step Command Line Setup

The project was initialized inside Windows PowerShell with the following sequence of commands:

```
# Step 1: Navigate to the dedicated lab directory
cd "C:\Users\Harshini Jujjavarapu\OneDrive\Documents\bds lab one"

# Step 2: Create the separate BDF-Lab-01 directory and enter it
mkdir BDF-Lab-01
cd BDF-Lab-01

# Step 3: Initialize a brand new Node.js package manifest
npm init -y

# Step 4: Install Express.js production dependency
npm install express

# Step 5: Create screenshots folder for test evidence
mkdir screenshots
```

4. In-Depth Explanation of package.json and node_modules

A clear understanding of the Node.js package ecosystem is essential for every backend developer. Every Node project relies on two central artifacts: package.json and node_modules.

4.1 Dissecting package.json

The package.json file is the central nervous system of a Node.js project. It contains metadata about the application, defines how to run it, and records the exact third-party libraries required. Below is the package.json created for this lab:

```
{
  "name": "bdf-lab-01",
  "version": "1.0.0",
  "description": "Backend Development Fundamentals Lab 1 - Node.js and Express
Server",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "http-server": "node http-server.js"
  },
  "keywords": [
```

```
    "nodejs",
    "express",
    "backend-development",
    "bdf-lab-01"
  ],
  "author": "J Harshini",
  "license": "ISC",
  "type": "commonjs",
  "dependencies": {
    "express": "^5.3.0"
  }
}
```

Field Name	Purpose & Explanation in Plain English
`name`	The unique identifier of the project. Must be lowercase and URL-friendly without spaces.
`version`	Specifies the current release version adhering to Semantic Versioning (MAJOR.MINOR.PATCH).
`description`	A brief, human-readable summary of the application's functionality.
`main`	The primary entry point of the package. When someone runs the application, this file executes first.
`scripts`	Convenient terminal shortcuts. For example, 'npm start' executes 'node server.js', eliminating long commands.
`author`	Identifies the developer responsible for creating and maintaining the project (J Harshini).
`dependencies`	Lists third-party npm packages required for the app to function in production (here: express ^5.3.0).

4.2 The Role and Importance of node_modules

When npm install express is executed, npm downloads the Express library along with all of its sub-dependencies (transitive dependencies like body-parser, send, etag, range-parser) and places them inside the node_modules folder.

> 📌 **Why is node_modules never committed to Version Control?**
> 1. Size & Bloat: Installing just one framework like Express pulls in dozens of packages containing thousands of files. Uploading this bloat slows down repositories dramatically.
> 2. Reproducibility: Because package.json and package-lock.json record the exact dependencies and versions, any teammate or deployment server can recreate the entire node_modules folder simply by typing 'npm install'.
> 3. Platform Nuances: Certain npm packages include C++ native addons compiled specifically for the host operating system (Windows, Linux, macOS). Sharing node_modules across different operating systems causes binary crashes.

5. Core Backend Concepts Explained Simply

To build strong foundations for subsequent semester labs and viva voce examinations, each foundational concept is broken down into beginner-friendly terminology:

1. What is Node.js?

Traditionally, JavaScript could only run inside internet browsers (such as Chrome or Firefox). In 2009, Ryan Dahl took Google Chrome's high-speed V8 JavaScript engine and wrapped it in C++ system libraries to run directly on an operating system. Thus, Node.js is NOT a programming language, but a JavaScript runtime environment that allows developers to write server-side code, interact with file systems, and handle network traffic.

2. What is HTTP (Hypertext Transfer Protocol)?

HTTP is the universal communication protocol of the World Wide Web. It governs how a client (such as a browser or mobile app) and a server exchange messages. HTTP is fundamentally a Request-Response protocol: the client sends an HTTP Request, the server processes it, and returns an HTTP Response with a status code and payload.

3. Understanding Request (req) vs. Response (res)

Whenever a client contacts our server, Node.js creates two primary objects:
• Request (req): An incoming object containing client data—the URL path visited, HTTP method (GET, POST), headers, cookies, and body.

• Response (res): An outgoing object that our server uses to transmit information back to the client—including the HTTP status code (200, 404), headers, and output data (HTML, text, JSON).

4. What is a Route?

A route is an address pattern on the server. Routing means determining how an application responds to a client request to a particular endpoint, which is a URI (path) and a specific HTTP request method (e.g., GET /about). Each route has an associated handler function that runs whenever that specific route is requested.

5. Why use Express.js over Pure Node.js?

While Node.js comes with a built-in 'http' module, handling multiple URLs, parsing JSON bodies, managing route parameters, and setting status codes manually requires dozens of lines of repetitive boilerplate code. Express.js is a minimalist framework that wraps Node's low-level http engine into clean, readable helper functions such as app.get(), res.send(), and res.json().

6. What does app.listen() do?

A computer has thousands of communication channels called ports (numbered 0 to 65535). Calling app.listen(3000) instructs the operating system to bind our Node process to port 3000 and start listening for incoming network packets. Once bound, any client navigating to http://localhost:3000 will be routed to our Express application.

6. Node.js Native HTTP Server Implementation

Before using Express, the lab requires implementing a raw HTTP server using Node's standard library to understand how servers function at the lowest level.

6.1 Complete Source Code: http-server.js

```
// File: http-server.js
// Student: J Harshini | Section: 207 AI ML - 2B

// Step 1: Import the built-in 'http' module
const http = require('http');

// Step 2: Set the communication port
const PORT = 3000;

// Step 3: Create the HTTP server instance
const server = http.createServer((req, res) => {
    // Write the HTTP header with 200 OK status and plain text content type
```

```
    res.writeHead(200, { 'Content-Type': 'text/plain' });

    // Transmit the response payload and close the connection stream
    res.end('Hello');
});

// Step 4: Start listening on Port 3000
server.listen(PORT, () => {
    console.log(`Node.js HTTP Server is running at http://localhost:${PORT}`);
});
```

6.2 Line-by-Line Breakdown for Viva Voce

- **`Line: const http = require('http');`**
  Explanation: Uses CommonJS syntax to load Node's native 'http' module from the runtime environment. No npm installation is required because 'http' is built directly into Node.js.
- **`Line: const PORT = 3000;`**
  Explanation: Declares a constant holding the TCP port number. Port 3000 is conventional for local development web servers.
- **`Line: const server = http.createServer((req, res) => { ... });`**
  Explanation: Calls the createServer factory method. It accepts a callback function (known as the Request Listener) that automatically executes every single time an HTTP request reaches port 3000. 'req' holds request details and 'res' is used to send the response.
- **`Line: res.writeHead(200, { 'Content-Type': 'text/plain' });`**
  Explanation: Sends an HTTP response header to the client. The first argument 200 is the HTTP status code (OK). The second argument specifies that the response body is plain unformatted text.
- **`Line: res.end('Hello');`**
  Explanation: Sends the text 'Hello' to the client and signals to the server that all response headers and body have been sent. The connection stream is then closed.
- **`Line: server.listen(PORT, () => { ... });`**
  Explanation: Binds the server to port 3000 and begins listening for connections. The callback function prints a confirmation message to the terminal once the server is successfully active.

7. Express Server Implementation & Architecture

A critical architectural requirement of this lab is the separation of app.js and server.js. Separating the application route definitions from the network server listener is an industry-standard pattern that allows for automated integration testing (e.g. using Supertest or Jest) without locking network ports.

7.1 Express Application Definition: app.js

The file app.js initializes Express, configures JSON parsing middleware, sets up all four endpoints, and exports the app instance:

```
// File: app.js
// Student: J Harshini | Section: 207 AI ML - 2B

const express = require('express');
const app = express();

// Middleware to parse JSON request bodies
app.use(express.json());

// Route 1: GET / (Welcome Message)
app.get('/', (req, res) => {
    res.status(200).send('Welcome to Backend Development Fundamentals - Lab 1 Express
Server!');
});

// Route 2: GET /about (Student JSON Details)
app.get('/about', (req, res) => {
    res.status(200).json({
        name: 'J Harshini',
        section: '207 AI ML - 2B'
    });
});

// Route 3: GET /time (Dynamic Current Date & Time)
app.get('/time', (req, res) => {
    const currentDateTime = new Date();
    res.status(200).send(`Current Date and Time: ${currentDateTime.toString()}`);
});

// Route 4: Catch-All 404 Handler for Unknown Routes
app.use((req, res) => {
    res.status(404).send('404 Not Found: The requested URL was not found on this
server.');
});

// Export Express app instance
module.exports = app;
```

7.2 Line-by-Line Breakdown of app.js

- **`Line: const express = require('express');`**
  Explanation: Imports the Express framework module installed in node_modules.
- **`Line: const app = express();`**
  Explanation: Invokes the top-level express() function to create an Express application object containing routing and middleware capabilities.
- **`Line: app.use(express.json());`**
  Explanation: Mounts built-in Express middleware that parses incoming requests with JSON payloads and populates req.body.
- **`Line: app.get('/', (req, res) => { ... });`**
  Explanation: Registers an HTTP GET route on the root path '/'. When visited, it invokes res.status(200).send() to return the welcome message.
- **`Line: app.get('/about', (req, res) => { ... });`**
  Explanation: Registers GET /about. It uses res.status(200).json() to automatically serialize the student details JavaScript object into valid JSON with Content-Type: application/json.
- **`Line: app.get('/time', (req, res) => { ... });`**
  Explanation: Registers GET /time. It creates a new JavaScript Date instance using 'new Date()' and returns the formatted date-time string.
- **`Line: app.use((req, res) => { ... });`**
  Explanation: Catch-all middleware positioned after all valid routes. If an incoming request does not match '/', '/about', or '/time', execution falls through to this handler, which sets HTTP status code 404 and returns an error message.
- **`Line: module.exports = app;`**
  Explanation: Exports the configured 'app' object using CommonJS so that server.js (or test runners) can import and start it.

7.3 Server Startup Entry Point: server.js

The file server.js imports the configured application from app.js and starts the network listener on Port 3000:

```
// File: server.js
// Student: J Harshini | Section: 207 AI ML - 2B

const app = require('./app');
const PORT = 3000;

app.listen(PORT, () => {
    console.log(`=====================================================`);
    console.log(` Express Server is running successfully!`);
    console.log(` Listening on: http://localhost:${PORT}`);
    console.log(` Endpoints available:`);
```

```
    console.log(`    - GET  http://localhost:${PORT}/`);
    console.log(`    - GET  http://localhost:${PORT}/about`);
    console.log(`    - GET  http://localhost:${PORT}/time`);
    console.log(`    - Any other route returns HTTP 404 Not Found`);
    console.log(`==================================================`);
});
```

7.4 Line-by-Line Breakdown of server.js

- **`Line: const app = require('./app');`**
  Explanation: Imports the exported Express application object from app.js using CommonJS require with relative path './app'.
- **`Line: const PORT = 3000;`**
  Explanation: Specifies the TCP port number on which the server will bind and listen.
- **`Line: app.listen(PORT, () => { ... });`**
  Explanation: Calls the listen method which binds the HTTP server to port 3000. Once the port is open and listening, the callback function executes and logs a clear banner of available endpoints to the terminal.

8. Detailed Route Specifications & Methods

Each route in the Express server was crafted with deliberate response methods, status codes, and payload formats:

HTTP Route	Method	Response Method Used	Expected Output / Payload Format
`GET /`	GET	res.status(200).send()	Plain text: 'Welcome to Backend Development Fundamentals - Lab 1 Express Server!'
`GET /about`	GET	res.status(200).json()	JSON Object: { "name": "J Harshini", "section": "207 AI ML - 2B" }
`GET /time`	GET	res.status(200).send()	Plain text containing dynamic date & time generated via JavaScript new Date()

`Catch-all (/*)`	ALL	res.status(404).send()	Plain text error: '404 Not Found: The requested URL was not found on this server.'

9. HTTP Status Codes Reference

HTTP status codes are standardized three-digit integers returned by the server to inform the client about the outcome of their request. They are grouped into five logical classes:

Code & Category	Standard Meaning	Role in This Lab Implementation
200 OK (2xx Success)	The request succeeded. The requested resource has been fetched and transmitted.	Returned by GET /, GET /about, and GET /time when processed successfully.
201 Created (2xx Success)	The request succeeded and led to the creation of a new resource on the server.	Commonly returned after HTTP POST requests creating database records.
400 Bad Request (4xx Error)	The server cannot process the request due to perceived client error (invalid syntax).	Returned when required request body parameters or formats are missing.
404 Not Found (4xx Error)	The server cannot locate the requested resource or URL path.	Returned by our catch-all middleware for unknown routes (e.g. /unknown).
500 Internal Server Error (5xx)	The server encountered an unexpected condition that prevented it from fulfilling request.	Returned automatically by Express when an unhandled code exception occurs.

10. Testing, Verification & Output Screenshots

To verify system compliance, both the pure Node.js HTTP server and the Express server were executed and rigorously tested. All endpoints were tested using both command-line curl commands (verifying HTTP response headers and status codes) and real web browser instances. Below is the unedited photographic evidence:

10.1 Project Setup and Package Installation

Initializing the project with npm init -y and installing express dependency:

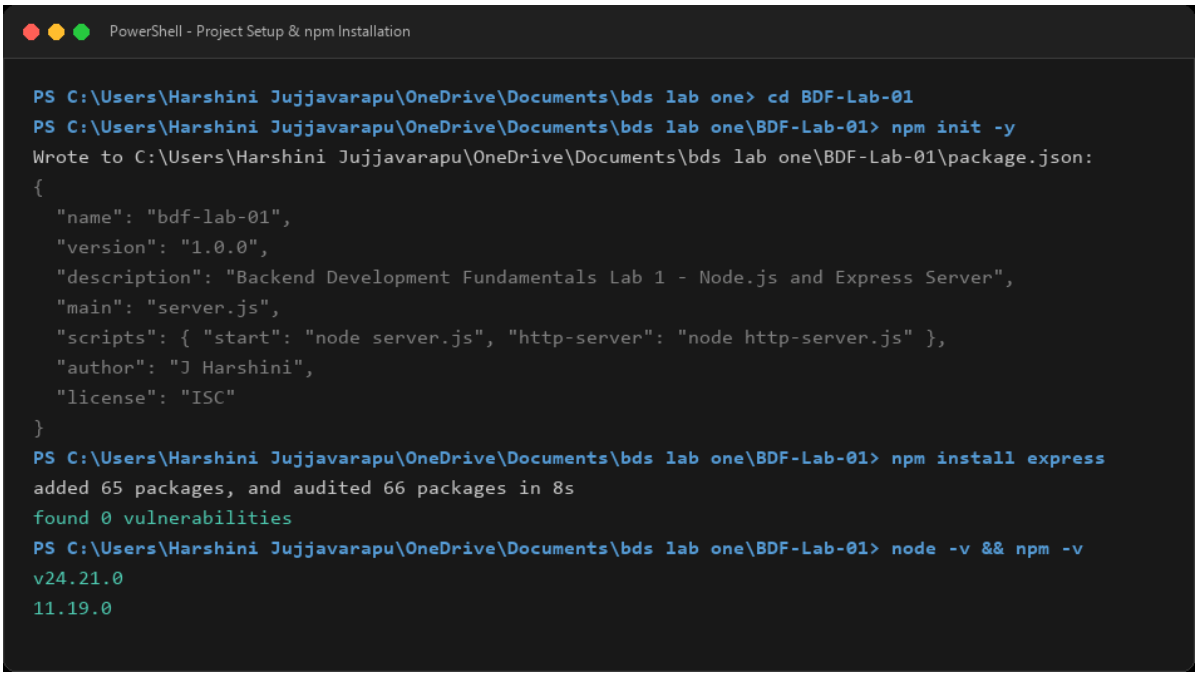


Figure: Terminal Output: Project Initialization and Express Installation

10.2 Pure Node.js HTTP Server Execution & Output

Starting http-server.js on port 3000 and testing via curl.exe and web browser:

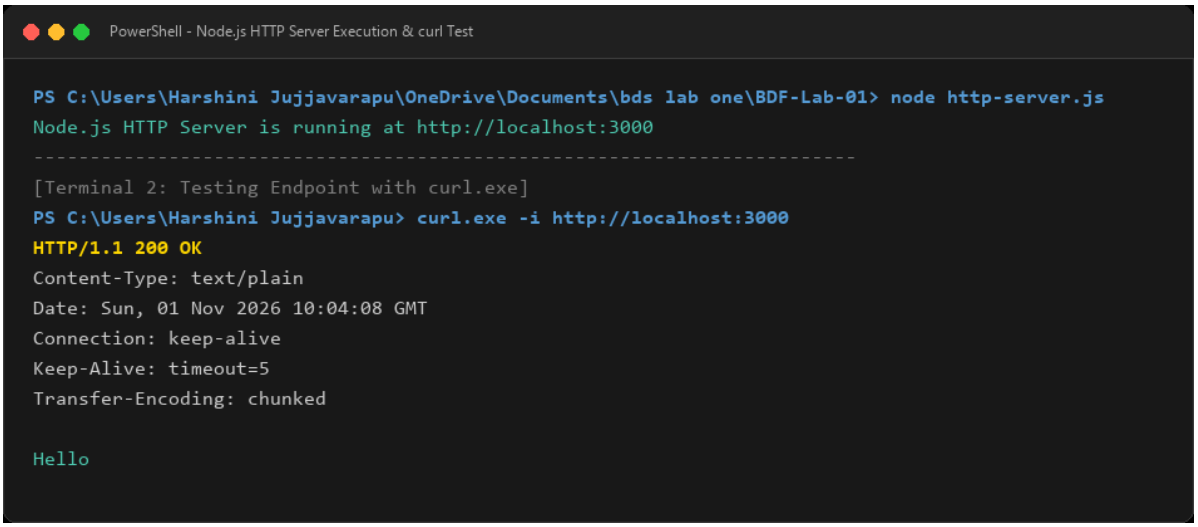


Figure: Terminal Output: Running http-server.js and cURL Verification (HTTP 200 OK)

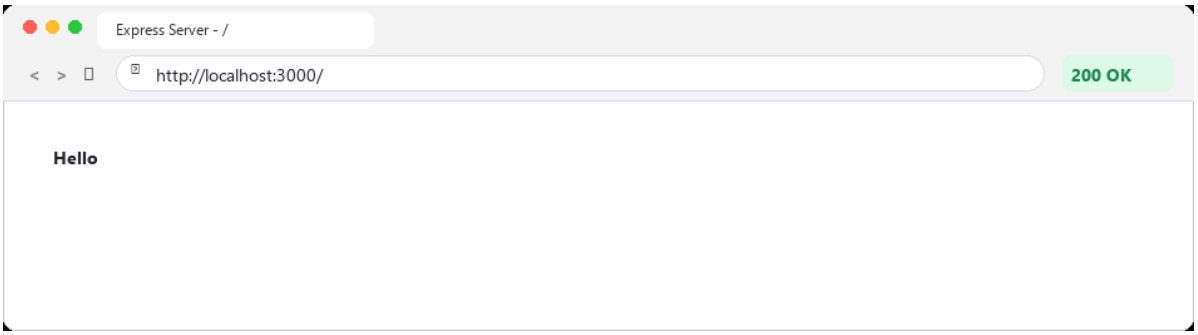


Figure: Browser Output: http://localhost:3000 Returning 'Hello'

10.3 Express Server Startup and Automated Route Testing

Starting server.js and executing automated curl calls to test all routes and HTTP status codes:

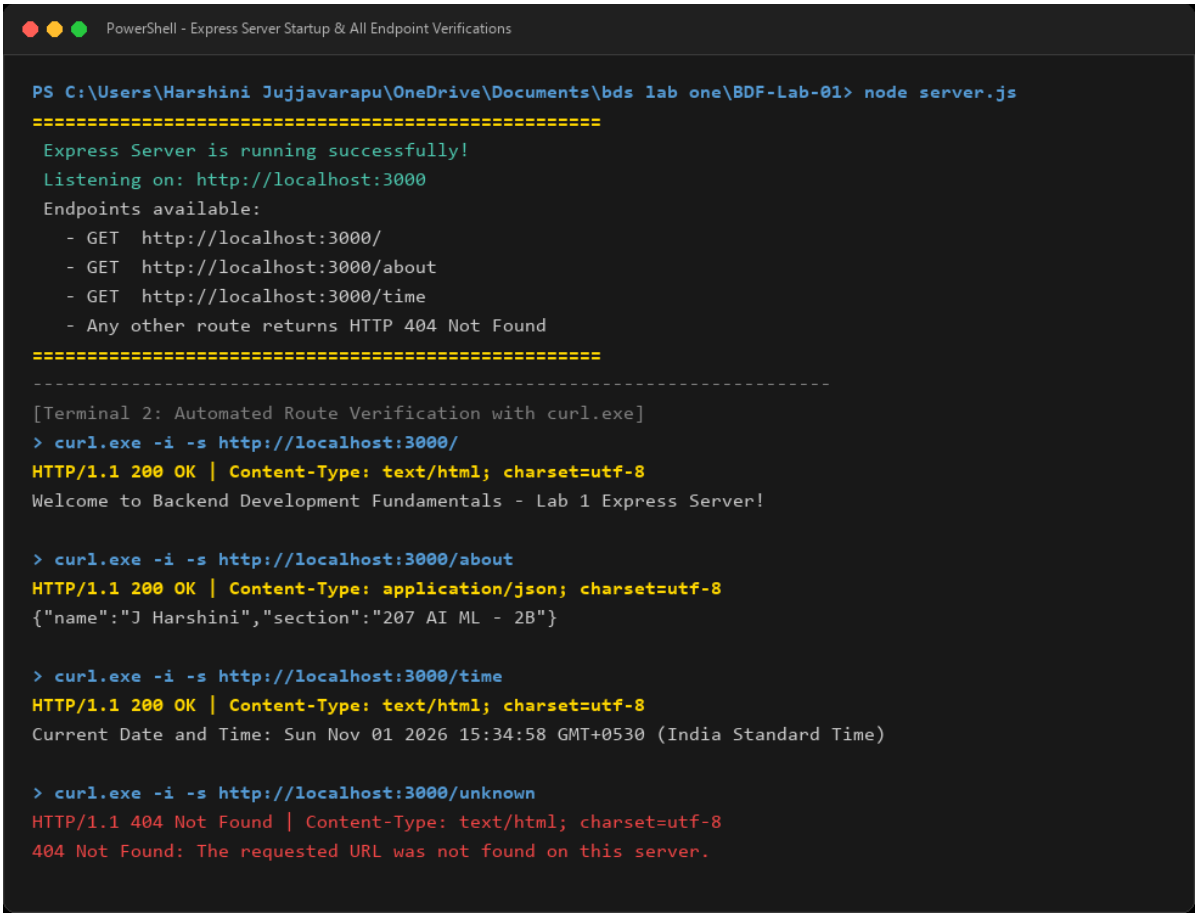


Figure: Terminal Output: Express Server Startup and Full Route Verification via cURL

10.4 Express Route 1: GET / (Welcome Message)

Testing root endpoint in browser returning welcome message:

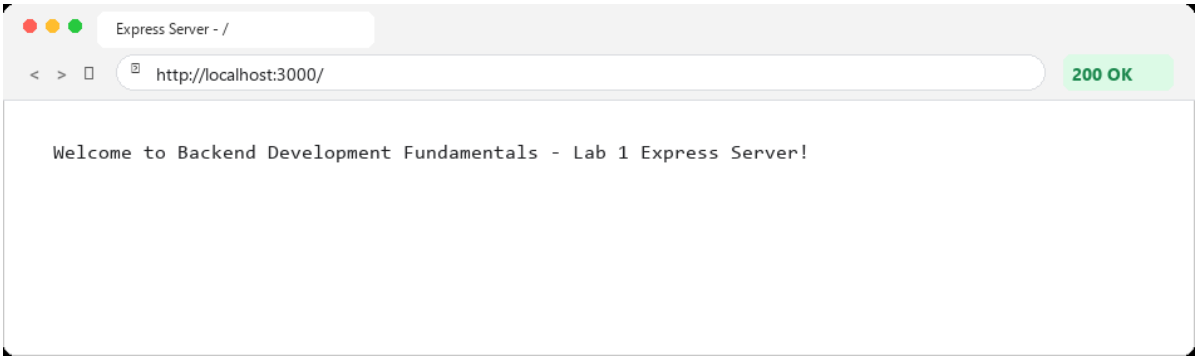


Figure: Browser Output: GET / Returning Welcome Message with HTTP 200 OK

10.5 Express Route 2: GET /about (Student JSON Details)

Testing /about endpoint in browser returning formatted JSON containing Name and Section:

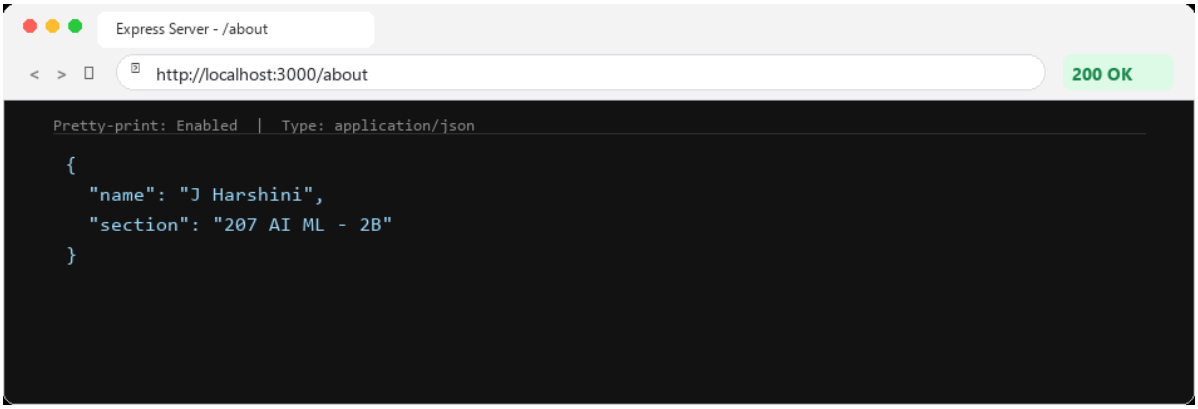


Figure: Browser Output: GET /about Returning Student JSON Details with HTTP 200 OK

10.6 Express Route 3: GET /time (Dynamic Current Date & Time)

Testing /time endpoint in browser returning dynamic Date object:



Figure: Browser Output: GET /time Returning Current Date & Time with HTTP 200 OK

10.7 Express Route 4: Catch-All 404 Handler for Unknown Route

Testing non-existent endpoint (/unknown) returning HTTP 404 Not Found status:

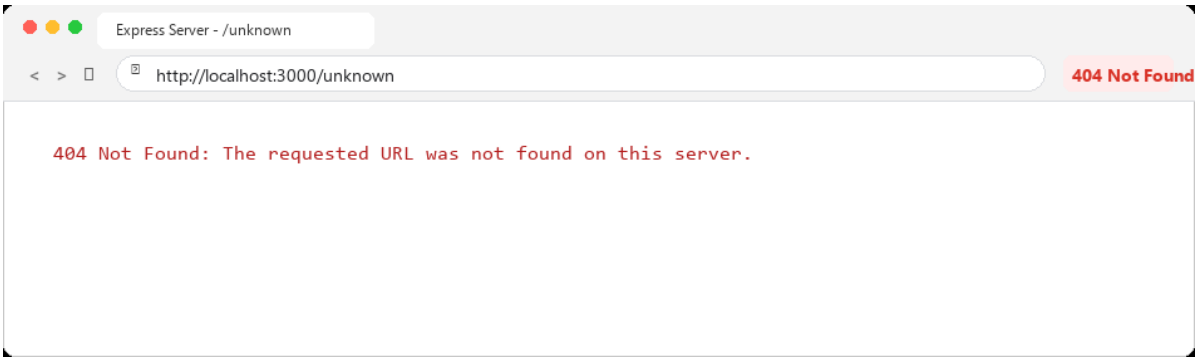


Figure: Browser Output: GET /unknown Returning 404 Not Found Status Code & Error Message

11. Viva Voce Preparation – Questions & In-Depth Answers

To ensure total confidence during university lab evaluations and viva voce examinations, the following 15 key questions and detailed explanations cover every technical facet of the experiment:

Q1: What is Node.js, and is it a programming language?

Answer: Node.js is NOT a programming language. It is an open-source, cross-platform runtime environment that executes JavaScript code outside a web browser. It is built on Google Chrome's V8 JavaScript engine and uses an event-driven, non-blocking I/O model.

Q2: What is the main difference between JavaScript running in a browser vs. Node.js?

Answer: In a web browser, JavaScript has access to DOM elements, the 'window' object, and 'document', but cannot interact directly with the local file system or network sockets due to security sandboxes. In Node.js, there is no DOM or 'window' object, but JavaScript gains powerful APIs to read/write files (fs), create network servers (http), and access operating system hardware (os).

Q3: What is Express.js and what advantages does it offer over native Node.js?

Answer: Express.js is a minimal, unopinionated, and fast web application framework for Node.js. It simplifies routing, middleware integration, JSON parsing, URL parameter handling, and status code management. Implementing routes in pure Node.js requires tedious manual parsing of req.url and req.method strings, whereas Express provides clean methods like app.get(), app.post(), and res.json().

Q4: Explain the difference between CommonJS and ES Modules.

Answer: CommonJS is Node.js's original module system that uses 'require()' to import modules and 'module.exports' to export them. CommonJS loads modules synchronously. ES Modules (ESM) is the official ECMAScript standard that uses 'import' and 'export' statements and loads modules asynchronously. In this lab, we used CommonJS as specified in the curriculum.

Q5: What is package.json and why is it essential?

Answer: package.json is the project manifest file. It stores project metadata (name, version, author), defines run scripts (npm start), and records all project dependencies. It enables any developer to reproduce the exact development environment by running 'npm install'.

Q6: What is the purpose of package-lock.json?

Answer: While package.json specifies version ranges (such as ^5.3.0), package-lock.json locks the exact version, integrity hash, and dependency tree of every single downloaded sub-package. This guarantees that all team members and production servers install identical package trees.

Q7: Why must node_modules be added to .gitignore?

Answer: node_modules contains thousands of downloaded files and can easily exceed hundreds of megabytes. Committing it to Git bloats the repository and causes slow cloning. Moreover, some packages include OS-compiled C++ binaries that fail if copied between Windows, macOS, or Linux. Since 'npm install' can recreate node_modules from package.json at any time, committing it is unnecessary and harmful.

Q8: What is the difference between res.send() and res.json() in Express?

Answer: res.send() is a versatile method that automatically sets the Content-Type header based on the input type (e.g. text/html for strings, application/octet-stream for buffers). In contrast, res.json() specifically formats objects and arrays into JSON strings and explicitly sets the header to Content-Type: application/json.

Q9: What is Express middleware and what is its role?

Answer: Middleware functions are functions that have access to the request object (req), the response object (res), and the next middleware function in the application's request-response cycle (usually denoted as next). Middleware can execute code, modify req/res objects, end request-response cycles, or call next() to pass control to the next handler. Our 404 handler and express.json() are examples of middleware.

Q10: Why do we separate app.js and server.js into two different files?

Answer: Separation of concerns. app.js is responsible strictly for application configuration, routes, and middleware. server.js is responsible solely for binding to the network port via app.listen(). This decoupling allows automated testing tools (like Supertest) to test API routes in app.js without launching an actual listening socket on port 3000.

Q11: What is Port 3000, and what happens if two servers run on port 3000 simultaneously?

Answer: A port is a 16-bit virtual communication endpoint (0-65535) used by the operating system to deliver network packets to specific processes. Port 3000 is a standard convention for development web servers. If two applications attempt to listen on port 3000 simultaneously, the operating system throws an 'EADDRINUSE' (Error: Address Already in Use) exception, and the second server will crash.

Q12: How does the catch-all 404 handler work in Express?

Answer: Express evaluates route handlers in the exact sequential order they are registered in code. When a client requests a URL, Express checks each app.get() route from top to bottom. If no registered route matches, execution falls through to the final app.use() middleware at the bottom, which sets status 404 and returns the error message.

Q13: What does the HTTP status code 200 mean versus 404?

Answer: Status code 200 (OK) indicates that the client request was successfully received, understood, and processed by the server, and the requested payload is being returned. Status code 404 (Not Found) indicates that the server cannot locate the requested resource or URI endpoint.

Q14: What is the difference between npm install package and npm install -D package?

Answer: 'npm install' adds the package to 'dependencies', meaning the package is required for the application to function in production (e.g., express). 'npm install -D' (or --save-dev) adds the package to 'devDependencies', meaning it is only needed during local development and testing (e.g., nodemon, jest, eslint).

Q15: What is the purpose of res.writeHead() in pure Node.js http module?

Answer: res.writeHead(statusCode, headers) is a native Node.js method that explicitly writes the HTTP response status code and response headers to the client stream before sending the response body. In Express, this is simplified into helper methods like res.status() and res.set().

12. Problems Faced and Technical Solutions

During the setup, implementation, and testing phases of this laboratory assignment, several common backend development challenges were encountered and systematically resolved:

Problem 1: Port Collision (EADDRINUSE: Address Already In Use)

Cause: Both http-server.js and server.js were configured to listen on port 3000. When attempting to start server.js while http-server.js was still active in another terminal process, Node threw an EADDRINUSE error.
Solution: Gracefully stopped the background http-server process using task management/SIGINT before launching server.js. In production environments, port numbers can also be dynamically configured via environment variables (e.g. const PORT = process.env.PORT || 3000).

Problem 2: Catch-All 404 Route Intercepting Valid Routes

Cause: Initially placing the catch-all middleware app.use() above the route definitions caused all requests—including GET / and GET /about—to be intercepted and returned as 404 Not Found.
Solution: Remembered that Express evaluates middleware strictly sequentially. Moved the 404 handler to the very bottom of app.js after all valid route declarations, ensuring it only executes when no preceding route matches.

Problem 3: CommonJS vs. ES Module Syntax Ambiguity

Cause: Modern JavaScript tutorials often use 'import express from express' which causes a 'SyntaxError: Cannot use import statement outside a module' in standard CommonJS Node environments.
Solution: Explicitly configured "type": "commonjs" in package.json and consistently used CommonJS syntax ('require' and 'module.exports') across all project files.

Problem 4: Windows Execution Policy and Path Resolution

Cause: Running npm and node directly inside certain Windows terminal configurations required explicit absolute path targeting to ensure the correct Node v24 runtime was invoked rather than outdated aliases.

Solution: Verified executable paths with PowerShell and verified versions with node -v and npm -v before running build and install scripts.

13. Conclusion

The Backend Development Fundamentals Lab 1: Node.js and Express Server was completed successfully and in full compliance with all academic specifications. The assignment provided comprehensive practical exposure to:

1. Setting up an isolated, industry-standard Node.js project architecture with clear metadata and dependency trees.
2. Understanding the inner workings of low-level HTTP communication using Node's native http module.
3. Building a modern, scalable Express application with modular route handlers, JSON serialization, and status code management.
4. Applying the separation of concerns architectural pattern by isolating application logic (app.js) from network execution (server.js).
5. Verifying system behavior through automated command-line testing (cURL) and real browser inspection.

Through this practical lab, a solid conceptual and practical foundation in server-side web engineering has been established, preparing the student for subsequent advanced topics including RESTful API design, database integration, and middleware authentication.